

Visual Change Detection & Evidence Agent

(UI Change Monitoring using Computer-Using Agents)

1. Project Overview & Personal Understanding

Modern enterprise applications and public-facing websites undergo frequent UI updates. These changes may include content refreshes, layout shifts, styling adjustments, or component reordering. Through this project, I developed a clear understanding that many of these changes are **silent** - they do not always break functionality but can still impact usability, workflows, or compliance requirements.

The **Visual Change Detection & Evidence Agent** was built to address this gap. Instead of relying on backend APIs or DOM-level checks, the system observes applications the same way a human user would—by visually inspecting the interface. This approach aligns with the **Computer-Using Agent (CUA)** paradigm, where an agent perceives and reasons about a system through its graphical interface.

My goal was to design a solution that is **simple, reliable, and evidence-oriented**, while remaining adaptable to real enterprise environments such as those at Eli Lilly.

2. Problem Statement

From my understanding, UI validation today faces three key challenges:

- 1. Manual verification does not scale**
Teams often rely on screenshots, walkthroughs, or ad-hoc checks, which are time-consuming and inconsistent.
- 2. Traditional automation misses visual regressions**
DOM or API-based checks may pass even when visible UI changes negatively affect the user experience.
- 3. Lack of audit-ready evidence**
In regulated environments, it is often necessary to demonstrate *what changed* and *when*, using clear visual proof.

These challenges create a need for an automated system that can continuously monitor UI changes and generate trustworthy visual evidence.

3. How the Model Works (End-to-End Flow)

Step 1: Baseline Creation

- The agent opens the target URL using Microsoft Edge.
- A full-page screenshot is captured.
- This image becomes the baseline reference for future comparisons.

Step 2: Current State Capture

- On subsequent executions, the same page is opened again.
- A new screenshot is captured, representing the current UI state.

Step 3: Visual Difference Detection

- The baseline and current screenshots are normalized.
- A pixel-by-pixel comparison is performed.
- Changed regions are identified and highlighted.

Step 4: Evidence Generation

- An HTML report is generated automatically.
- The report displays:
 - Baseline UI
 - Current UI
 - Visual diff with highlighted changes
- A change percentage is calculated to quantify UI variation.

This flow ensures that every detected change is **explainable, reviewable, and traceable**.

4. Key Design Decisions & My Reasoning

Why Screenshot-Based Comparison

I intentionally chose screenshot comparison because it reflects **exactly what the end user sees**. This avoids false negatives caused by invisible DOM changes and ensures relevance to real user experience.

Why Real Browser Execution

Using Microsoft Edge allowed the solution to operate in a realistic enterprise setup without requiring custom browser installations. This improves portability and adoption in corporate environments.

Why Evidence-First Reporting

Rather than logging raw differences, the system produces a visual HTML report. This makes the output usable not only for developers, but also for QA, compliance, and business stakeholders.

5. Impact and Relevance to Lilly

From my understanding of enterprise and regulated environments, this solution can provide meaningful value to Eli Lilly in several ways:

Operational Efficiency

- Reduces manual UI validation efforts.
- Saves time during release cycles and routine checks.

Risk Reduction

- Detects unintended UI regressions early.
- Prevents silent changes from impacting users or workflows.

Compliance & Audit Support

- Generates visual, time-stamped evidence.
- Supports audit and validation requirements where proof of UI state is needed.

Scalability

- Can be extended to internal dashboards, portals, and regulated applications.

Can be integrated into scheduled monitoring or CI/CD pipelines in the future.

8. Learning Outcomes & Personal Contribution

Through this project, I gained hands-on experience in:

- Designing a solution around the **Computer-Using Agent** concept.
- Translating a real business problem into an automated system.
- Balancing technical correctness with enterprise usability.
- Building evidence-driven outputs rather than raw technical logs.

This project strengthened my understanding of how automation can be applied not just for testing, but for **continuous monitoring, compliance, and assurance**.

9. Conclusion

The **Visual Change Detection & Evidence Agent** demonstrates how Computer-Using Agents can autonomously observe user interfaces, reason about visual differences, and produce actionable evidence. By focusing on what users actually see, the solution bridges the gap between traditional automation and real-world UI monitoring - making it especially relevant for large, regulated organizations like Lilly.